



اُنِيْوَرْسِيْٓتِيْٓ تِيْكَنُوْلُوْجِيْٓ مَآرَا
UNIVERSITI
TEKNOLOGI
MARA

FACULTY OF INFORMATION SCIENCE

BACHELOR OF INFORMATION SCIENCE (HONS)

INFORMATION SYSTEM MANAGEMENT (CDIM262)

ADVANCED DATABASE MANAGEMENT (IMS560)

GROUP ASSIGNMENT:

TUITION DATABASE SYSTEM

PREPARED BY:

NAME	STUDENT ID
NURUL IZZATI BINTI MD. NIZAM	2024764955
ADRIANNA BATRISYA BINTI ABDUL HALIM	2024959971
NURFATIAH ASYIRA BINTI ABU KAHAR	2024776633
NORMISHA SAFIAH BINTI OMAR	2024963059

PREPARED FOR:

DR. ABDURRAHMAN BIN JALIL

DATE:

18 FEBRUARY 2026

ACKNOWLEDGEMENT

First and foremost, we would like to express our deepest gratitude to **Allah SWT** for granting us good health, strength, and patience throughout the completion of this tuition system assignment.

We would like to extend our sincere appreciation to our lecturer Dr. Abdurrahman Bin Jalil for the guidance, support, and valuable knowledge shared during this course. The continuous encouragement and constructive feedback greatly helped us in understanding the requirements of the assignment and improving the quality of our work.

We would also like to thank our group members for the excellent cooperation, commitment, and teamwork shown throughout the development of this tuition system.

Lastly, we are grateful to our friends and family for their moral support and understanding during this assignment period. Their encouragement motivated us to stay focused and complete this project to the best of our ability.

We sincerely hope that this tuition system fulfills the objectives of the assignment and demonstrates our understanding of the concepts learned throughout this course.

TABLE OF CONTENT

1.0 Introduction	2 - 3
1.1 Project Background	
1.2 Motivation	
1.3 Scope	
1.4 Tools and Technologies	
2.0 System Analysis	4 - 5
2.1 Problem Statement	
2.2 Intended Users	
2.3 Functional Requirements	
2.4 Non-Functional Requirements	
3.0 System Design	6 - 8
3.1 System Architecture	
3.2 Entity Relationship Diagram	
3.3 Interface Design	
4.0 Database Implementation	9 - 17
4.1 Database Schema Implementation	
4.2 SQL Table Creation	
4.3 Data Dictionary	
4.4 CRUD Operations	
4.5 Data Integrity and Constraints	
4.6 Security Measures	
5.0 System Testing	18 - 25
5.1 Testing Objectives	
5.2 Test Environment	
6.0 Conclusion	26
7.0 Limitations	27
8.0 Future Improvements	28

1.0 Introduction

1.1 Project Background

Tuition centres usually manage the information of the students, tutor assignments, class schedules and bookings by using manual methods such as physical copies, paper records, receipts or even spreadsheets. As students multiply by time, these traditional methods have become inefficient to keep as storages or archives as it will take space in the office, or get spilled on or go missing with no trace. It will become hard to maintain. Handling data manually can lead to issues like data redundancy, booking conflicts, records that are hard to confirm if it is accurate or not and also delays in administrative and registration process.

To address these challenges, we have created a system for the tuition management and class booking system, as a web based application integrated with a relational database. The system centralized all tuition-related information which allows administrators to manage the records efficiently while providing the students with an online platform to view and book one-by-one classes with the tutors or book an extra class.

1.2 Motivation

We found motivation and inspiration from the inefficiencies that are commonly found in traditional management systems that often rely on manual records and basic spreadsheets. Even though this method is prone to errors, duplicated data, complicating booking sessions especially when the student list keeps getting longer, insufficient tutors and class increases. Managing schedules and student bookings manually can also place a burden on the administration staff and reduce operational efficiency.

This project aims to develop a centralized, data-based driven management system that automatically helps the tuition management process. By integrating a relational database with a web-based application, the system will improve the data accuracy, consistency and efficiency. The system will provide a convenient platform for administrators to manage the records efficiently while allowing students to book classes easily which can enhance the overall user experience and practicing the practical application of advanced database management.

1.3 Scope

The scope of this project covers the development of a web-based application Tuition Management and Class Booking System that supports two main user roles: Admin and Student. The administrator will be responsible for managing the core system data such as student profiles, tutor information, subjects, classes and bookings through full CRUD operations. This system also allows administrators to monitor booking records and manage class assignments to ensure data consistency and operational efficiency.

As for the students, the system will provide functionality to view available classes and make class bookings online. The students can also view their one-by-one bookings or extra classes and its status through the system. The project focuses on implementing essential database driven features such as user authentication, data validation and relational integrity. Advanced features like online payment gateways, automated notifications and mobile application support are not included yet in this application but will be considered for future enhancements.

1.4 Tools and Technologies

This web application for tuition system was developed using a combination of web development and database technologies to ensure functionality, reliability and ease of use. PHP was used as the backend programming language to handle system logic and database interactions. The frontend of the system was developed by using HTML, CSS and also Bootstrap incorporated into creating a responsive and user friendly interface that can be accessed through a web browser.

MySQL was selected as the database management system with its strong support for relational databases, the implementation of data integrity and the PHP compatibility. Foreign keys constraints were implemented to maintain relationships between tables and tested over the Laragon local server environment which provides an integrated Apache server and MySQL database. phpMyAdmin also was used for database administration and query management, while Visual Studio Code was used to serve as the primary code editor for the development and debugging of the system.

2.0 System Analysis

2.1 Problem Statement

Many tuition centres still rely on manual or semi-manual methods like paper records and spreadsheets in order to manage student records, class schedules and bookings. These approaches are inefficient and become increasingly difficult to manage as the number of students and classes grows. The manual data management often leads to data redundancy, booking conflicts, inaccuracy of records and delays in administrative processes. Students often face challenges due to restricted access to current class information, which makes the booking process cumbersome and susceptible to mistakes.

To address this, a centralized, database-powered platform is required to streamline the management of tuition data and automate class reservations. Implementing such a system would enhance data reliability, minimize administrative effort, and significantly improve the overall experience for both students and administrators.

2.2 Intended Users

The system serves two primary user groups:

- a) **administrators**
- b) **and students.**

Administrators handle the essential operations, such as updating student records, tutor details, subjects, class schedules, and booking information. Their role is to ensure that all data remains accurate, consistent, and current.

Students, on the other hand, are the main users who access the system to browse available tuition classes and make reservations. With this platform, they can easily book classes online and check their booking history and status, eliminating the need for manual interaction with administrators.

2.3 Functional Requirements

Administrator Requirements:

- Administrators can add, edit, view, and delete student records.
- They can manage tutor details, subjects, and class timetables.
- They can access and oversee student booking information.
- The system ensures data accuracy through validation and relational rules.

Student Requirements:

- Students can log in securely with their own emails
- They can browse available tuition classes.
- They can book classes online.
- They can check their booking history and current booking status
- They can pay for booking their class

2.4 Non-Functional Requirements

These requirements define the system's quality standards and operational constraints.

- **Security:** The system must use session-based authentication, ensuring only authorized users can access functions according to their roles.
- **Usability:** It should provide a responsive, intuitive interface that is simple for both administrators and students to navigate.
- **Performance:** The system must handle requests and database operations efficiently to maintain smooth performance.
- **Data Integrity:** Relational integrity must be enforced through primary and foreign key constraints to prevent invalid or inconsistent data.
- **Reliability:** The system should manage errors effectively, displaying clear messages when invalid actions occur.

3.0 System Design

3.1 System Architecture

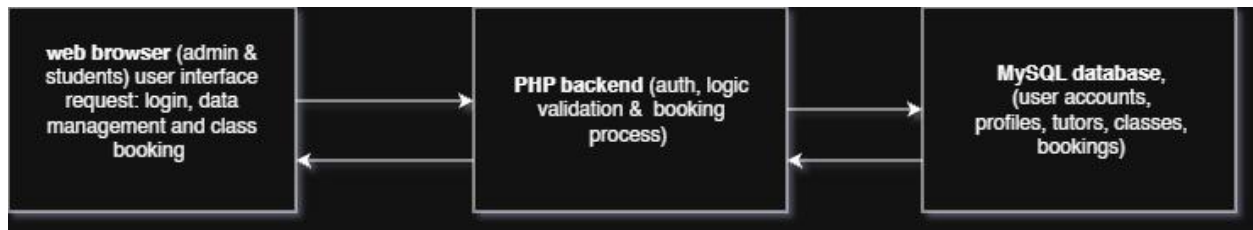
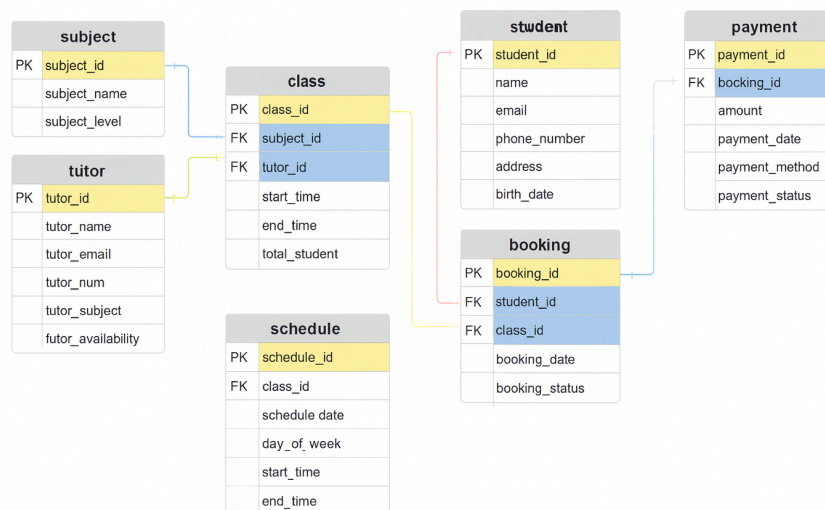


Figure 1: The diagram of system architecture

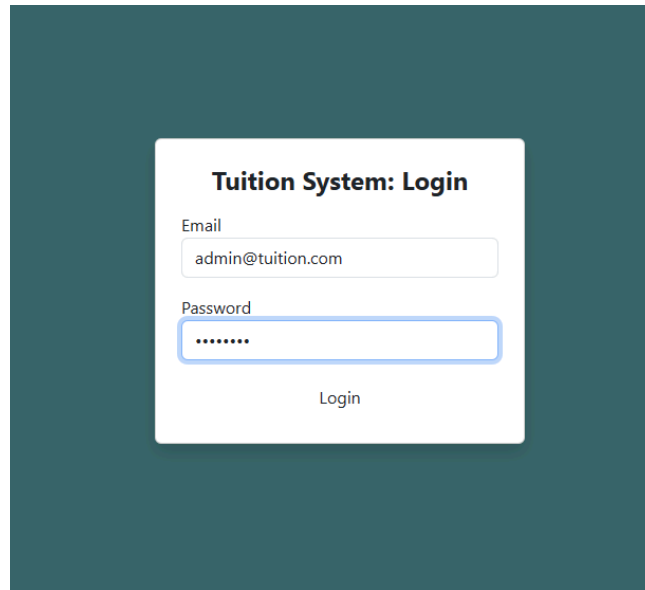
The system adopts a **client-server model**, ensuring a clear separation between the user interface, application logic, and data storage. Administrators and students interact with the system via a web browser, where requests such as login, data management, and class booking are processed by a PHP-based backend. The backend is responsible for authentication, validation, and business logic, and it communicates directly with a **MySQL database**.

The database maintains all essential records, including user accounts, student details, tutor information, class schedules, and booking data. This architecture enhances **maintainability, scalability, and security** by centralizing data handling and enforcing controlled access to stored information.

3.2 Entity Relationship Diagram (ERD)



3.3 Interface Design



The image shows a login interface for a 'Tuition System'. It consists of a dark teal background with a white login card centered on it. The card has the title 'Tuition System: Login' in bold. Below the title, there are two input fields: 'Email' with the value 'admin@tuition.com' and 'Password' with masked characters '*****'. A 'Login' button is located at the bottom of the card.

Figure 2: Login Interface

The user interface design is designed with simplicity and usability in mind by using a clean and responsive design layout that is developed with HTML, CSS and Bootstrap to ensure accessibility across different screen sizes. The main colours of the system are mostly a tone of blue. We also give separate interfaces for admin and students.

Admin are presented with a dashboard that provides management modules such as student management, tutor management, class management, booking records and payment records. Students are provided with pages to view available classes and review their booking history. Interface layouts were designed to support intuitive navigation and easy to use for administrations and students to adapt with just simple navigation. The interface can be improved in time, as the users adapt to the application and get used to them.

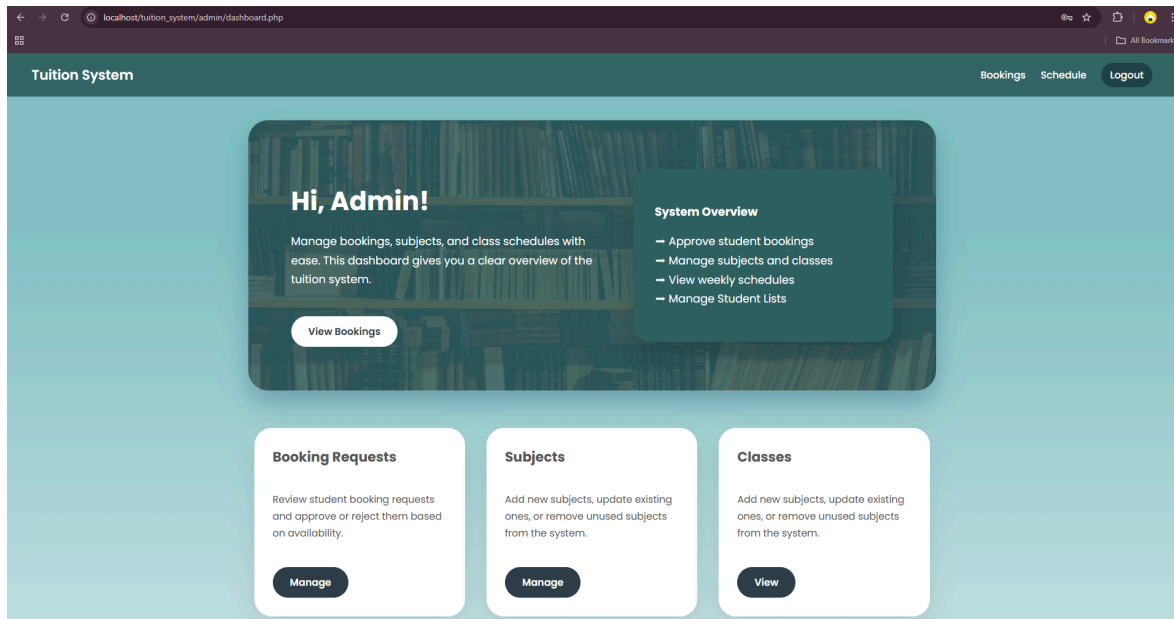


Figure 3: Admin Dashboard

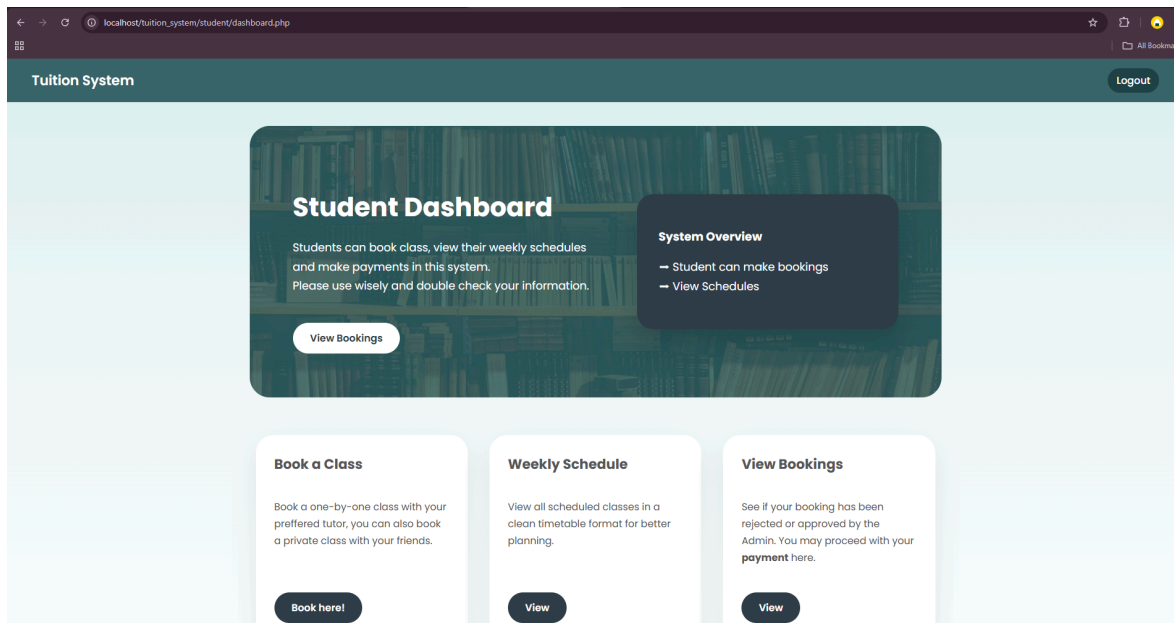


Figure 4: Student Dashboard

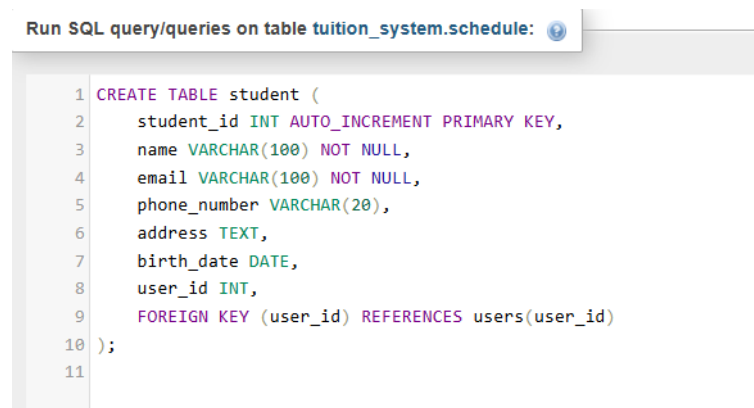
4.0 Database Implementation

4.1 Database Schema Implementation

The Tuition Management and Class Booking System is built on a relational database using MySQL. Its schema was developed in alignment with system requirements and the Entity Relationship Diagram (ERD). Each table corresponds to a distinct entity such as users, students, tutors, subjects, classes, and bookings with primary keys ensuring unique identification of records and foreign keys establishing inter-table relationships to uphold referential integrity.

Foreign key constraints play a crucial role in maintaining data consistency. For instance, every booking entry must reference a valid student and class, thereby preventing the storage of incomplete or disconnected records.

4.2 SQL Table Creation

A screenshot of the phpMyAdmin SQL editor interface. At the top, a header bar reads "Run SQL query/queries on table tuition_system.schedule:". Below this, a text area contains SQL code for creating a table named 'student'. The code is as follows:

```
1 CREATE TABLE student (  
2     student_id INT AUTO_INCREMENT PRIMARY KEY,  
3     name VARCHAR(100) NOT NULL,  
4     email VARCHAR(100) NOT NULL,  
5     phone_number VARCHAR(20),  
6     address TEXT,  
7     birth_date DATE,  
8     user_id INT,  
9     FOREIGN KEY (user_id) REFERENCES users(user_id)  
10 );  
11
```

Figure 5: Creating a table using SQL in phpMyAdmin

SQL scripts were used to create database tables with appropriate data types and constraints. Similar SQL scripts were implemented for other tables such as tutor, subject, class, and booking, each designed to support the required system functionality.

Like the figure above, we are creating a table student with student_id as its primary key (PK) with an auto-increment, which means the data will start at 1 and continue its numbers. Then we have the name VARCHAR (100) NOT NULL, email VARCHAR (100) NOT NULL, and others. For the foreign key, (user_id) it will be referenced back to its table and entity = users(user_id).

4.3 Data Dictionary

The data dictionary describes the structure of the tuition booking system database and defines each table and its attributes to ensure consistency and clarity in data management. The student table stores student personal information such as name, email, phone number, address, and birth date, with student_id serving as the primary key. User authentication and roles are managed through the users table, which contains login credentials and distinguishes between admin and student roles.

Academic information is organized using the subject and tutor tables. The subject table records subject names and levels, while the tutor table stores tutor details including name, email, contact number, subject expertise, and availability. These are linked to the class table, which represents scheduled classes by associating tutors and subjects along with class start time, end time, and total student capacity.

Student participation is handled through the booking table, where each booking links a student to a specific class and records the booking date, booking status, and payment status. Financial transactions are managed in the payment table, which stores payment amount, payment date, method, and status, and is connected directly to booking records. Lastly, the schedule table manages class scheduling information including class date, day of week, start and end times, and approved booking references, allowing both students and administrators to view confirmed sessions in the system calendar.

Primary keys uniquely identify records in each table, while foreign keys establish relationships between students, bookings, classes, tutors, subjects, payments, and schedules. This structured design supports booking management, payment tracking, tutor assignment, and timetable generation, ensuring efficient operation of the tuition management system.

tuition_system**booking**

Column	Type	Null	Default	Comments
booking_id (<i>Primary</i>)	int	No		
student_id	int	No		
class_id	int	No		
booking_date	date	Yes	<i>NULL</i>	
booking_status	varchar(50)	Yes	<i>NULL</i>	
payment_status	varchar(20)	Yes	Pending	

Indexes

Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
PRIMARY	BTREE	Yes	No	booking_id	5	A	No	
idx_booking_student	BTREE	No	No	student_id	2	A	No	
idx_booking_class	BTREE	No	No	class_id	1	A	No	

class

Column	Type	Null	Default	Comments
class_id (<i>Primary</i>)	int	No		
subject_id	int	No		
tutor_id	int	No		
start_time	time	Yes	<i>NULL</i>	
end_time	time	Yes	<i>NULL</i>	
total_student	int	Yes	<i>NULL</i>	

Indexes

Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
PRIMARY	BTREE	Yes	No	class_id	2	A	No	
fk_class_subject	BTREE	No	No	subject_id	2	A	No	
fk_class_tutor	BTREE	No	No	tutor_id	2	A	No	

payment

Column	Type	Null	Default	Comments
payment_id (<i>Primary</i>)	int	No		
booking_id	int	No		
amount	decimal(8,2)	No		
payment_date	date	Yes	<i>NULL</i>	
payment_method	varchar(50)	Yes	<i>NULL</i>	
payment_status	varchar(50)	Yes	<i>NULL</i>	

Indexes**Data Dictionary 1**

Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
PRIMARY	BTREE	Yes	No	payment_id	2	A	No	
idx_payment_booking	BTREE	No	No	booking_id	2	A	No	

schedule

Column	Type	Null	Default	Comments
schedule_id (<i>Primary</i>)	int	No		
class_id	int	Yes	NULL	
schedule_date	date	Yes	NULL	
day_of_week	varchar(20)	Yes	NULL	
start_time	time	Yes	NULL	
end_time	time	Yes	NULL	
booking_id	int	Yes	NULL	

Indexes

Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
PRIMARY	BTREE	Yes	No	schedule_id	4	A	No	
class_id	BTREE	No	No	class_id	1	A	Yes	
fk_schedule_booking	BTREE	No	No	booking_id	1	A	Yes	

student

Column	Type	Null	Default	Comments
student_id (<i>Primary</i>)	int	No		
name	varchar(100)	No		
email	varchar(100)	Yes	NULL	
phone_number	varchar(20)	Yes	NULL	
address	text	Yes	NULL	
birth_date	date	Yes	NULL	
user_id	int	Yes	NULL	

Indexes

Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
PRIMARY	BTREE	Yes	No	student_id	3	A	No	
email	BTREE	Yes	No	email	3	A	Yes	
idx_student_email	BTREE	No	No	email	3	A	Yes	

subject

Column	Type	Null	Default	Comments
subject_id (<i>Primary</i>)	int	No		
subject_name	varchar(100)	No		
subject_level	varchar(100)	No		

Data Dictionary 2

Indexes

Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
PRIMARY	BTREE	Yes	No	subject_id	5	A	No	

tutor

Column	Type	Null	Default	Comments
tutor_id (<i>Primary</i>)	int	No		
tutor_name	varchar(100)	No		
tutor_email	varchar(100)	Yes	NULL	
tutor_num	varchar(20)	Yes	NULL	
tutor_subject	varchar(100)	Yes	NULL	
tutor_availability	varchar(100)	Yes	NULL	

Indexes

Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
PRIMARY	BTREE	Yes	No	tutor_id	4	A	No	
tutor_email	BTREE	Yes	No	tutor_email	4	A	Yes	

users

Column	Type	Null	Default	Comments
user_id (<i>Primary</i>)	int	No		
name	varchar(100)	Yes	NULL	
email	varchar(100)	Yes	NULL	
password	varchar(255)	Yes	NULL	
role	enum('admin', 'student')	Yes	NULL	

Indexes

Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
PRIMARY	BTREE	Yes	No	user_id	2	A	No	
email	BTREE	Yes	No	email	2	A	Yes	
idx_user_role	BTREE	No	No	role	2	A	Yes	

Data Dictionary 3

4.4 CRUD Operations

CRUD represents the four fundamental operations applied to data within a database environment:

```
Run SQL query/queries on database tuition_system: ⓘ

1 INSERT INTO student (name, email, phone_number, address, birth_date)
2 VALUES ('Ali Ahmad', 'ali@gmail.com', '0123456789', 'KL', '2003-05-10');
3
```

CREATE

```
✓ Showing rows 0 - 1 (2 total, Query took 0.0018 seconds.)

SELECT * FROM student;
```

READ

```
1 UPDATE student
2 SET phone_number = '0198887777'
3 WHERE student_id = 1;
4
```

UPDATE

```
1 DELETE FROM student
2 WHERE student_id = 1;
3
4
```

DELETE

- **Create** – the insertion of new records into the database.
- **Read** – the retrieval and display of existing data.
- **Update** – the modification of stored records to reflect changes.
- **Delete** – the removal of records that are no longer required.

Collectively, these operations provide a structured and efficient means of managing data, ensuring both accuracy and reliability. Within this project, CRUD functionality has been implemented to oversee the management of students, tutors, classes, and bookings. This guarantees proper data handling and supports the overall effectiveness of the system.

The system provides complete support for CRUD (Create, Read, Update, Delete) operations across all key data entities. Through the web interface, administrators are able to add, view, modify, and remove records related to students, tutors, classes, and subjects. Students, meanwhile, can generate booking entries by selecting available classes and access their booking history for reference.

Typical CRUD activities include adding new bookings, retrieving existing booking details, updating booking statuses, and removing records when allowed. These processes are implemented using PHP in conjunction with MySQL queries, ensuring smooth and reliable communication between the application and the database.

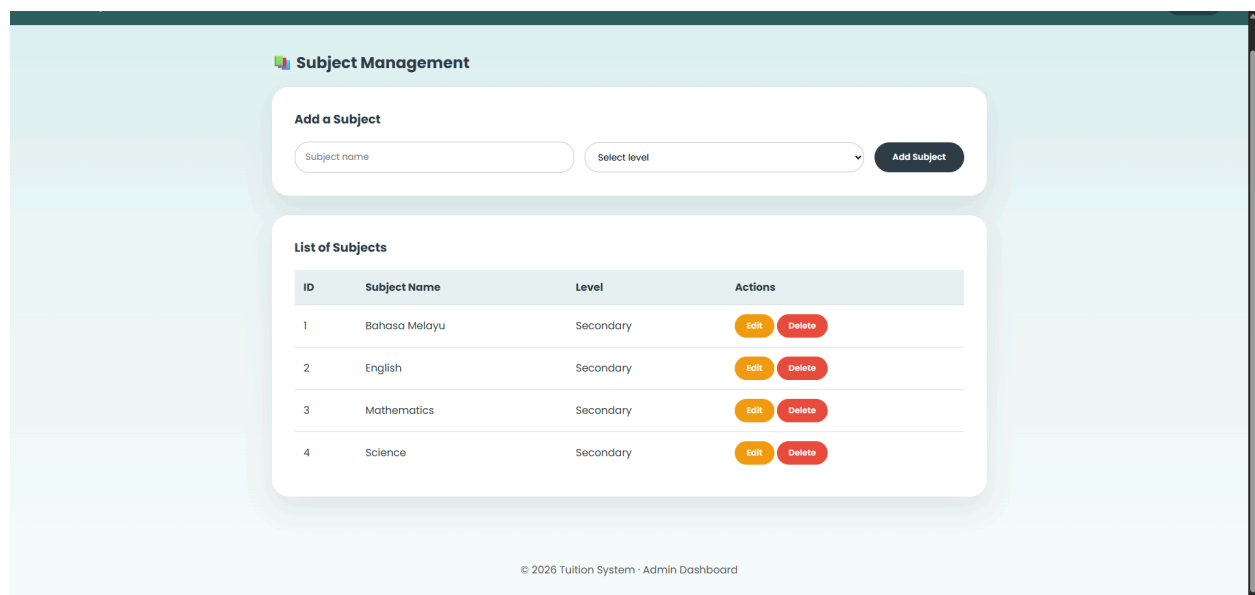


Figure 6: CRUD in tuition system, Subject Management.

4.5 Data Integrity and Constraints

Data integrity within the Tuition Management and Class Booking System is safeguarded through a combination of primary keys, foreign keys, and validation rules, all of which work together to ensure that the information stored in the database remains accurate, consistent, and reliable. Primary keys uniquely identify each record, preventing duplication and ambiguity, while foreign keys establish clear relationships between tables and enforce referential integrity.

This means that dependent records cannot exist without valid parent entries; for example, a student who has active bookings cannot simply be deleted unless those associated booking records are properly addressed, thereby avoiding orphaned or invalid data. Beyond database-level constraints, the system also incorporates input validation at the application layer, ensuring that users cannot submit incomplete or incorrect information.

This dual-layer approach combining structural database rules with application-level checks provides a robust safeguard against errors, enhances the reliability of stored data, and ultimately supports the smooth functioning of the entire system.

4.6 Security Measures

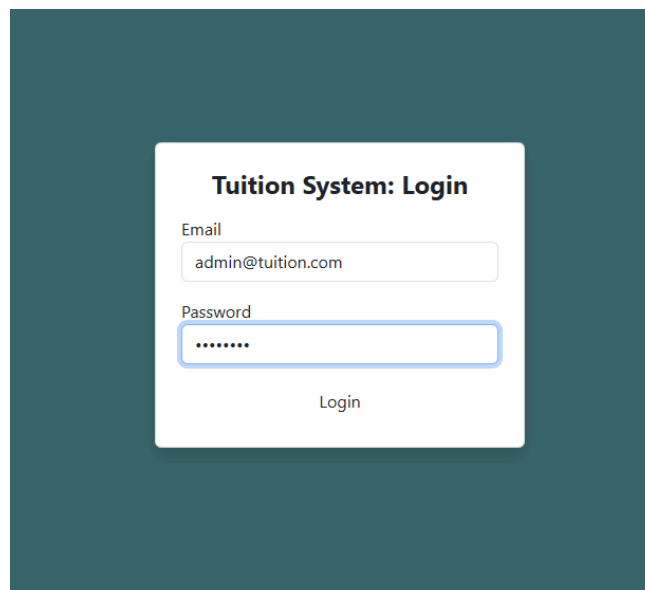
A screenshot of a login form titled "Tuition System: Login" centered on a dark teal background. The form is white with rounded corners and contains two input fields: "Email" with the value "admin@tuition.com" and "Password" with masked characters "*****". A blue border highlights the password field. Below the fields is a "Login" button.

Figure 7: Password Hash

Several security measures were implemented to protect the database and system data. Session-based authentication is used to restrict access based on user roles, ensuring that only authorized users can perform specific operations. User inputs are validated and sanitized to reduce the risk of SQL injection and data manipulation. Additionally, database access is handled through controlled server-side scripts, preventing direct unauthorized interaction with the database.

Security plays a critical role in the Tuition Management and Class Booking System, as it safeguards both user information and system data from unauthorized access or misuse. To achieve this, the system employs session-based authentication, requiring users to log in with valid credentials before gaining access. Permissions are strictly controlled according to user roles, such as administrator or student, ensuring that sensitive administrative functions and confidential records remain inaccessible to unauthorized individuals.

In addition to authentication, the system incorporates role-based access control to further strengthen protection. Administrators are granted the ability to manage core entities including students, tutors, classes, and bookings, while students are limited to viewing available classes and making reservations. Each page enforces role verification before granting access, thereby minimizing the risk of privilege escalation or misuse of system functions.

At the database level, integrity and security are reinforced through the use of primary keys, foreign keys, and relational constraints. These mechanisms prevent the creation of invalid records for example, bookings cannot be made for students or classes that do not exist and restrict the deletion of parent records that are still referenced elsewhere. This ensures that the database remains consistent and reliable across all operations.

Furthermore, the system applies input validation and sanitization to all user-provided data, reducing vulnerabilities such as SQL injection and preventing the submission of incomplete or malformed information. Error-handling routines are also in place to provide clear feedback to users without revealing sensitive details about the system's internal workings.

Taken together, these layered security measures provide comprehensive protection, uphold data integrity, and guarantee that the system functions dependably in a multi-user environment.

5.0 System Testing

5.1 Testing Objectives

The primary objective of system testing is to verify that the Tuition Management and Class Booking System operates as intended and fully complies with the defined requirements. Testing is carried out to ensure that each individual module performs its designated functions without errors and that all interactions with the database are executed correctly. This includes confirming that CRUD (Create, Read, Update, Delete) operations are implemented effectively for critical entities such as students, tutors, classes, and bookings, thereby supporting accurate and efficient data management.

A further objective of testing is to validate the enforcement of user role restrictions. The system must guarantee that administrators and students are only able to access features appropriate to their roles. For instance, students should be restricted from accessing administrative functions, while administrators should retain complete authority over data management tasks. Testing scenarios were designed to confirm that unauthorized access attempts are consistently blocked, ensuring that role-based access control is properly maintained.

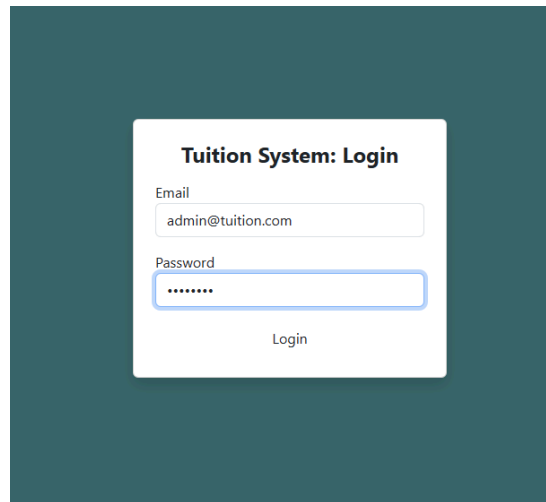
Another key focus of system testing is the preservation of data integrity and consistency within the database. This involves verifying that foreign key constraints prevent the creation of invalid relationships, such as bookings linked to non-existent students or classes, and ensuring that record deletions or modifications do not violate relational rules. Error-handling mechanisms were also tested to confirm that the system responds gracefully to invalid actions, providing clear and informative feedback to users rather than exposing system-level errors.

5.2 Test Environment

The system was tested in a local development environment by using the Laragon server which includes Apache and MySQL. Testing it was harsh as it was needed to perform using a web browser in order to simulate real user interaction. PhpmyAdmin was used to verify database records to ensure the data was stored and updated correctly.

5.3 Test Cases

LOG IN AS ADMINISTRATOR



The image shows a login form titled "Tuition System: Login" centered on a dark teal background. The form is white and contains two input fields: "Email" with the value "admin@tuition.com" and "Password" with masked characters "*****". Below the fields is a "Login" button.

Figure 8: Logging in with Admin info

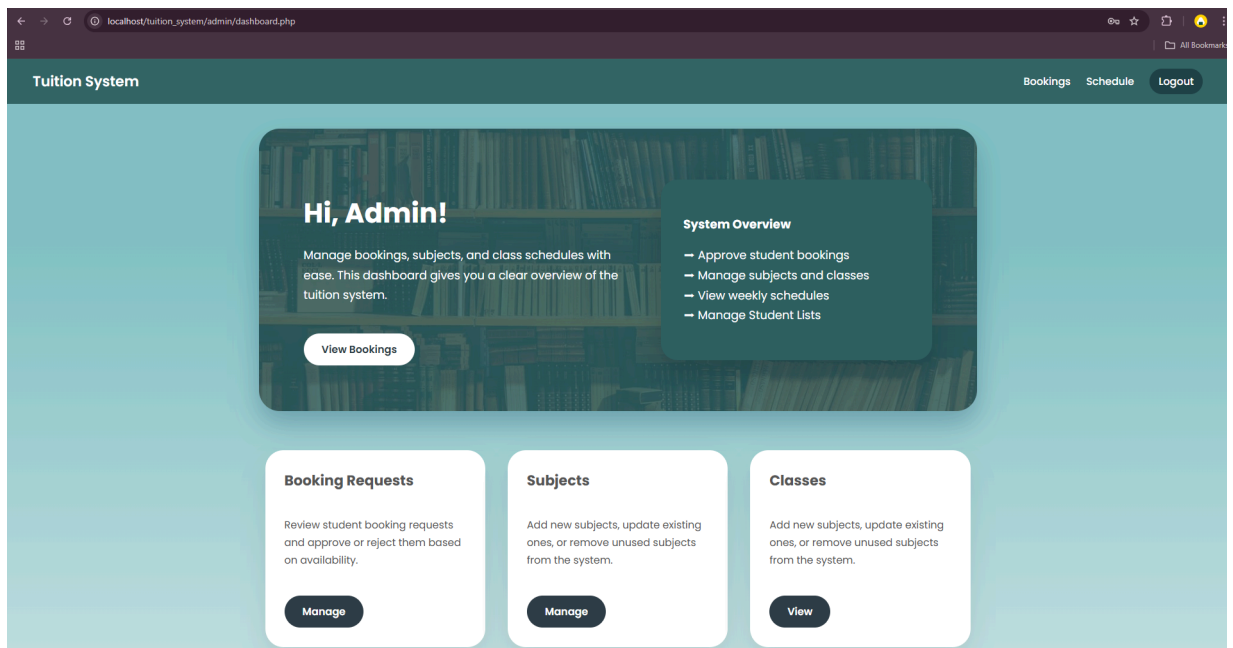


Figure 9: Main Page

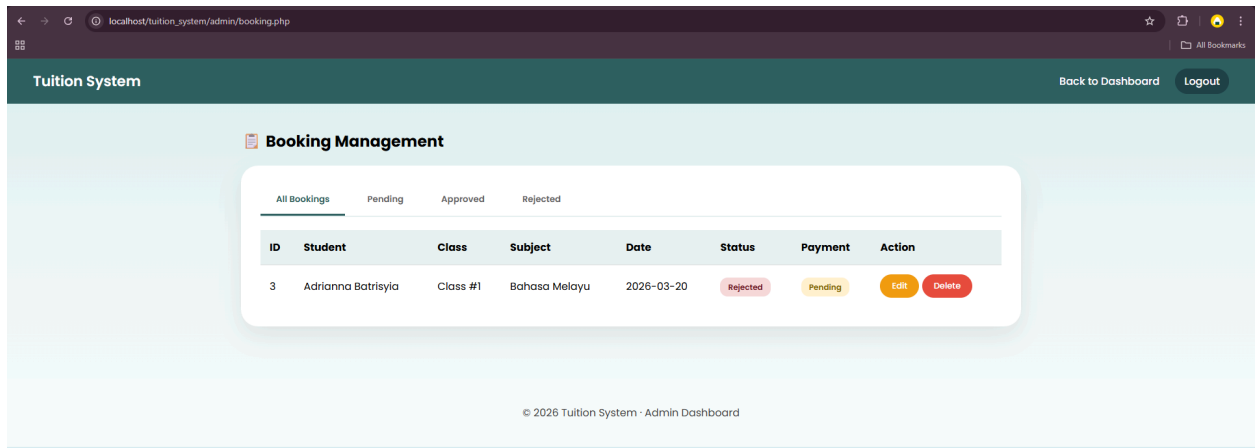


Figure 10: Booking Management page

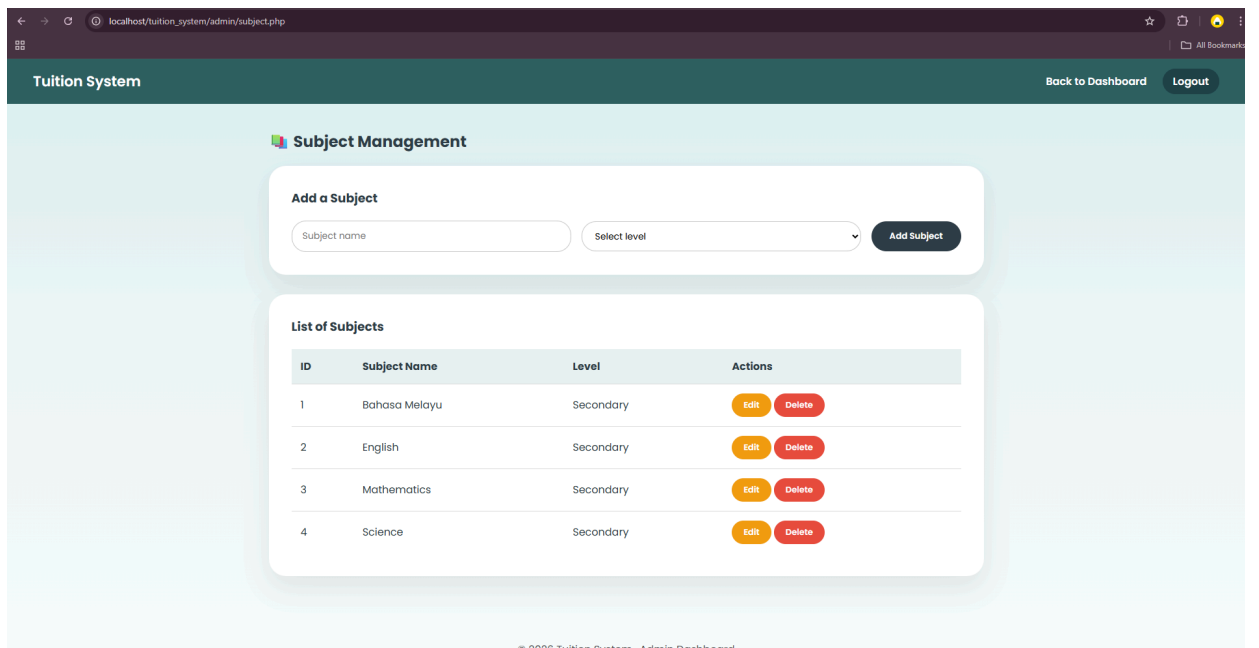


Figure 11: Subject Management page

← → 🔄 localhost/tuition_system/admin/student.php ☆ 📖 All Bookmarks

★ **Note:** When you add a student, a login account is automatically created using their email.
Default Password: student123 (Students should change this after first login)

Add New Student

Full Name Email (Will be used for login)

Phone Number Date of Birth

Address

Add Student & Create Login

Student List

ID	Name	Email (Login)	Phone	Birth Date	Has Login	Action
1	Nazarul Hafizi	nazarulhafizi@student.com	011 2341 2312	2002-11-10	✓ Yes	Edit Delete
3	Adrianna Batrisyia	adrianna@student.com	011 2613 8587	2002-02-10	✓ Yes	Edit Delete

Figure 14: Student List

← → 🔄 localhost/tuition_system/admin/tutor.php ☆ 📖 All Bookmarks

Tuition System [Back to Dashboard](#) [Logout](#)

Tutor Management

Add Tutor

Tutor Name Email

Phone Number Subject Expertise

Availability

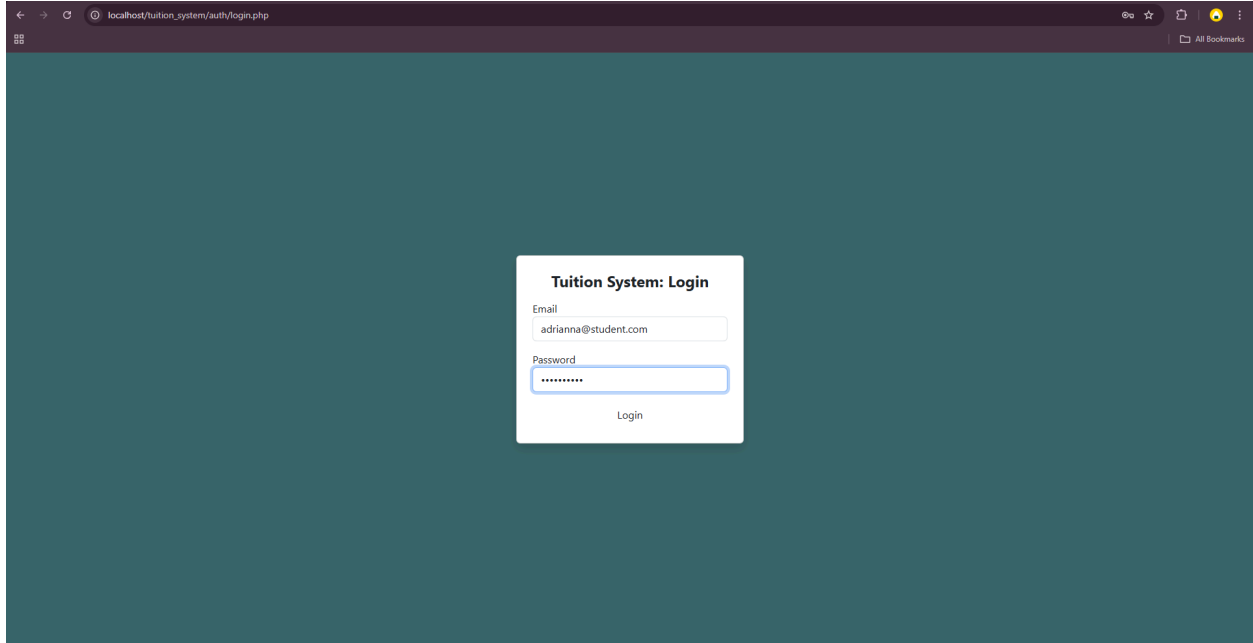
Add Tutor

Tutor List

ID	Name	Email	Phone	Subject	Availability	Action
1	Nurul Izzati bin Md Nizam	nurulizzati@tutor.com	012 111 3333	English	Monday - Friday	Edit Delete
			019 992			Edit

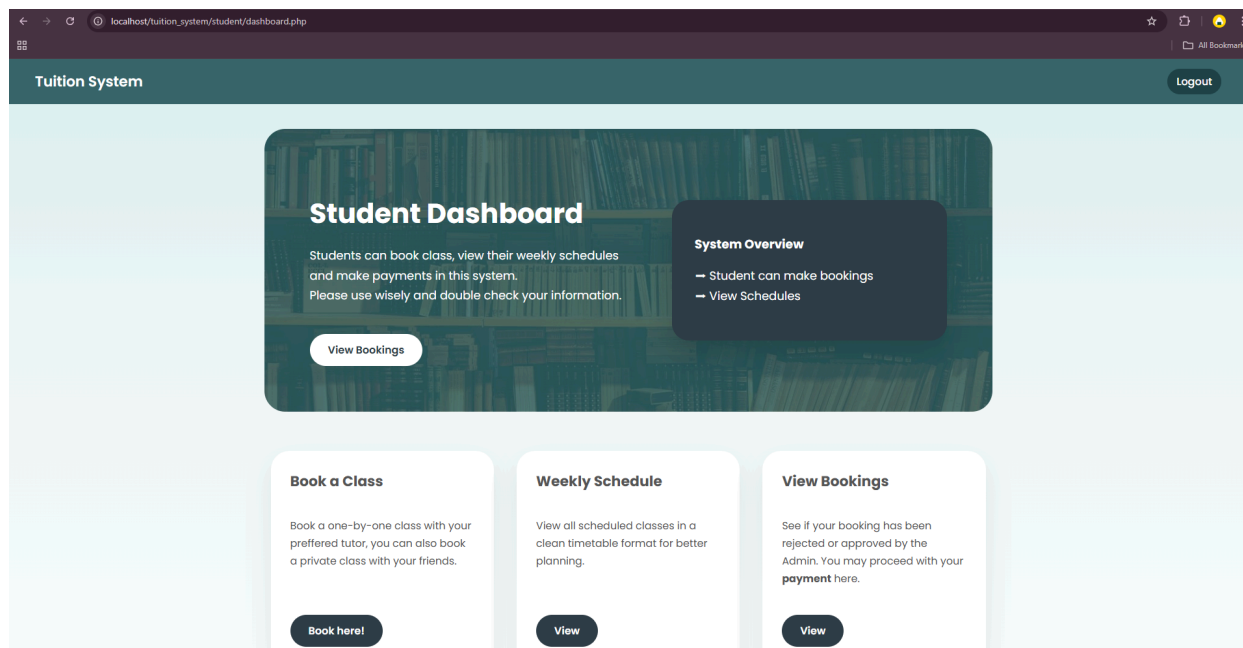
Figure 15: Tutor Management

LOG IN AS STUDENT EMAIL



The screenshot shows a web browser window with the address bar displaying 'localhost/tuition_system/auth/login.php'. The page has a dark teal background. In the center, there is a white login form titled 'Tuition System: Login'. The form contains two input fields: 'Email' with the value 'adrianna@student.com' and 'Password' with masked characters '*****'. Below the password field is a 'Login' button.

Figure 16: Log in with Student email info



The screenshot shows the 'Tuition System' student dashboard. The header includes the title 'Tuition System' and a 'Logout' button. The main content area features a large 'Student Dashboard' section with a bookshelf background. This section includes a description: 'Students can book class, view their weekly schedules and make payments in this system. Please use wisely and double check your information.' and a 'View Bookings' button. To the right is a 'System Overview' box with two items: 'Student can make bookings' and 'View Schedules'. Below this are three white cards: 'Book a Class' with a 'Book here!' button, 'Weekly Schedule' with a 'View' button, and 'View Bookings' with a 'View' button. The 'View Bookings' card contains text about booking status and a link to 'payment'.

Figure 17: Student Dashboard

The screenshot shows a web browser window with the URL `localhost/tuition_system/student/book_class.php`. The page has a dark green header with 'Tuition System' on the left and 'Back to Dashboard' and 'Logout' on the right. The main content area is light blue and features a white card titled 'Book a Class'. Inside the card, there is a message: 'Select a class. Your booking will be sent for admin approval.' Below this, there are two sections: 'Select Class' with a dropdown menu showing 'Choose Class', and 'Select Date' with a text input field showing 'mm/dd/yyyy' and a calendar icon. At the bottom of the card is a dark green button labeled 'Submit Booking'. At the very bottom of the page, there is a small copyright notice: '© 2026 Tuition System · Student Dashboard'.

Figure 18: Book Class for Student

The screenshot shows a web browser window with the URL `localhost/tuition_system/student/calendar.php`. The page has a dark green header with 'Tuition System' on the left and 'Dashboard' and 'Logout' on the right. The main content area is light blue and features a white card titled 'My Class Schedule'. Below the title, it says 'February 2026' and has a 'today' button and navigation arrows. The calendar is a grid with days of the week as columns and dates as rows. The date '14' is highlighted in yellow. The calendar is a dummy schedule with no actual class bookings.

Sun	Mon	Tue	Wed	Thu	Fri	Sat
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
1	2	3	4	5	6	7
8	9	10	11	12	13	14

Figure 19: Calendar for Student (dummy)

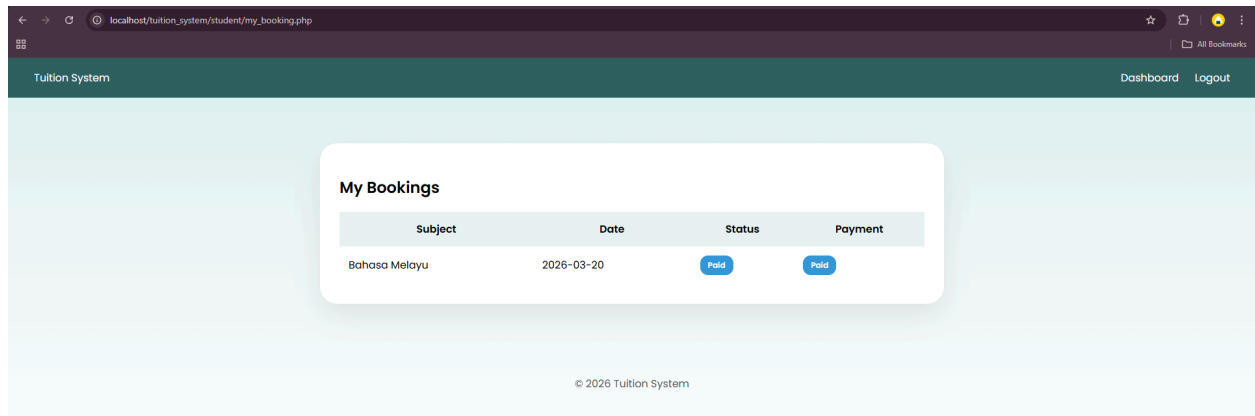


Figure 20: Booking Review and Payment

6.0 Conclusion

The development of the Tuition Database System successfully addresses the inefficiencies and challenges associated with traditional manual record-keeping methods used by tuition centres. By implementing a centralized, database-driven platform, this system has demonstrated the practical application of advanced database management concepts, including relational database design, data normalization, referential integrity, and CRUD operations.

Throughout this project, we achieved several key objectives. First, we designed and implemented a robust relational database schema using MySQL that effectively manages students, tutors, subjects, classes, bookings, payments, and schedules. The use of primary and foreign keys ensures data integrity and prevents inconsistencies such as orphaned records or invalid references. Second, we developed a user-friendly web-based interface using PHP, HTML, CSS, and Bootstrap that provides distinct functionalities for administrators and students, ensuring appropriate access control through role-based authentication.

The system successfully implements comprehensive CRUD operations, allowing administrators to efficiently manage all aspects of the tuition centre while enabling students to book classes conveniently online. Testing confirmed that the system functions as intended, with proper validation of user inputs, enforcement of data constraints, and graceful handling of errors. The implementation of session-based authentication and input sanitization provides a secure environment for both administrative operations and student interactions.

This project has provided valuable hands-on experience in database design, implementation, and management. We gained practical knowledge in translating real-world requirements into a functional database schema, implementing business logic through server-side scripting, and ensuring data security and integrity. The integration of frontend and backend components demonstrated the importance of systematic planning, iterative development, and thorough testing in creating reliable information systems.

While the current implementation successfully meets the core requirements outlined in this project, there remain opportunities for enhancement and expansion. The system establishes a solid foundation upon which additional features can be built to further improve functionality, usability, and scalability. Overall, this Tuition Database System serves as an effective solution for modernizing tuition centre operations and demonstrates our understanding of the principles and practices taught in the Advanced Database Management course.

7.0 Limitations

Despite successfully meeting the core objectives, the current implementation of the Tuition Management and Class Booking System has several limitations that should be acknowledged:

- **Manual Payment Processing:** The system currently tracks payment status manually through administrator updates. It lacks integration with online payment gateways such as PayPal, Stripe, or local banking systems, which would automate payment processing and provide immediate payment confirmation to students.
- **Absence of Automated Notifications:** The system does not include automated email or SMS notifications for important events such as booking confirmations, class reminders, payment reminders, or schedule changes. Users must manually check the system for updates, which may lead to missed communications or scheduling conflicts.
- **Limited Reporting Capabilities:** While the system maintains comprehensive records, it lacks advanced reporting and analytics features. Administrators cannot easily generate detailed reports on student enrollment trends, tutor performance, revenue analysis, or class utilization rates without manual data extraction and analysis.

8.0 Future Improvements

To address the limitations identified and further enhance the system's functionality, the following improvements are recommended for future development:

- **Online Payment Integration:** Integrate with popular payment gateways such as Financial Process Exchange (FPX), Bank QR or local banking APIs to enable automated online payment processing. This would allow students to pay for classes directly through the system, with automatic update of payment status and generation of digital receipts.
- **Automated Notification System:** Implement an email and SMS notification system to automatically inform users about booking confirmations, upcoming classes, schedule changes, payment reminders, and cancellations. This would improve communication efficiency and reduce the likelihood of missed classes or payment deadlines.
- **Advanced Reporting and Analytics:** Develop comprehensive reporting features that allow administrators to generate detailed analytics on student enrollment trends, tutor performance metrics, revenue reports, class attendance rates, and popular subjects. Visual dashboards with charts and graphs would facilitate data-driven decision-making.
- **Integrated Communication Platform:** Add an internal messaging system that enables direct communication between students and tutors, including support for file attachments, assignment submissions, and feedback exchange. This would create a more comprehensive learning management environment within the system.
- **Attendance Tracking:** Implement an attendance management module that allows tutors to mark student attendance for each class session. This would provide valuable data for monitoring student engagement and could be integrated with payment adjustments for missed classes.